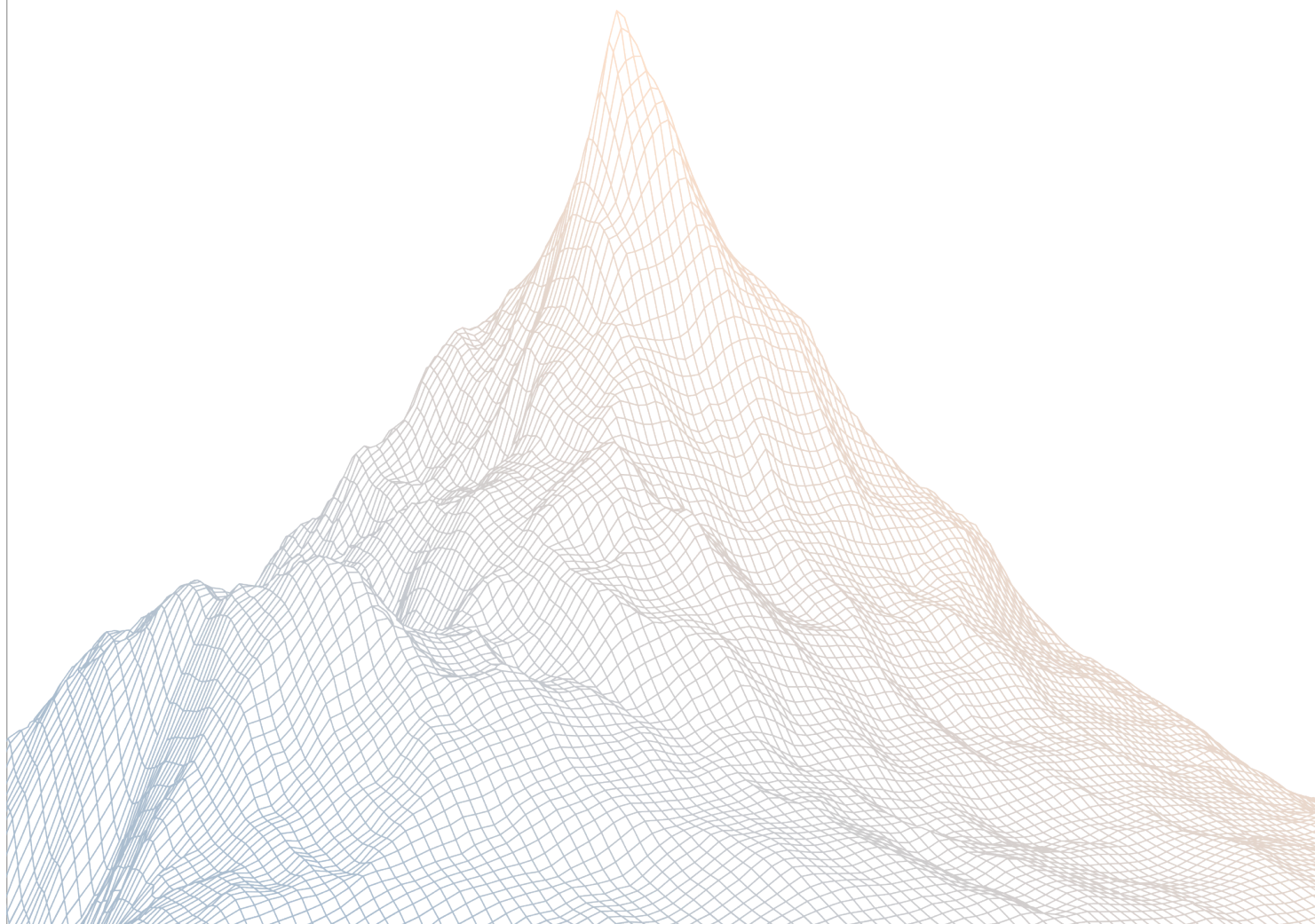


Berachain

Smart Contract Security Assessment

VERSION 1.1



AUDIT DATES:

May 15th to May 19th, 2026

AUDITED BY:

adriro
ether_sky

Contents

1	Introduction	2
1.1	About Zenith	3
1.2	Disclaimer	3
1.3	Risk Classification	3
2	Executive Summary	3
2.1	About Berachain	4
2.2	Scope	4
2.3	Audit Timeline	5
2.4	Issues Found	5
3	Findings Summary	5
4	Findings	6
4.1	High Risk	7
4.2	Medium Risk	11
4.3	Informational	17

1

Introduction

1.1 About Zenith

Zenith assembles auditors with proven track records: finding critical vulnerabilities in public audit competitions.

Our audits are carried out by a curated team of the industry's top-performing security researchers, selected for your specific codebase, security needs, and budget.

Learn more about us at <https://zenith.security>.

1.2 Disclaimer

This report reflects an analysis conducted within a defined scope and time frame, based on provided materials and documentation. It does not encompass all possible vulnerabilities and should not be considered exhaustive.

The review and accompanying report are presented on an "as-is" and "as-available" basis, without any express or implied warranties.

Furthermore, this report neither endorses any specific project or team nor assures the complete security of the project.

1.3 Risk Classification

SEVERITY LEVEL	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

2

Executive Summary

2.1 About Berachain

Berachain is a high-performance EVM-Identical Layer 1 blockchain utilizing Proof-of-Liquidity (PoL) and built on top of the modular EVM-focused consensus client framework BeaconKit.

2.2 Scope

The engagement involved a review of the following targets:

Target	contracts-staking-pools
Repository	https://github.com/berachain/contracts-staking-pools/pull/99
Commit Hash	7c5167af3e47dd1992236ff075ce0529b20fff01
Files	Diff from de5b000f5bb9d850793f11c26acdad41ec393a6d src/**/*.sol
Target	Mitigation Review
Repository	https://github.com/berachain/contracts-staking-pools/pull/99
Commit Hash	f003dcf82cac6aa67f74cba1d8906eb5940b6361
Files	Diff from de5b000f5bb9d850793f11c26acdad41ec393a6d src/**/*.sol

2.3 Audit Timeline

May 15, 2026	Audit start
May 19, 2026	Audit end
May 21, 2026	Report published

2.4 Issues Found

SEVERITY	COUNT
Critical Risk	0
High Risk	2
Medium Risk	2
Low Risk	0
Informational	6
Total Issues	10

3

Findings Summary

ID	Description	Status
H-1	SmartOperator upgrade corrupts storage layout	Resolved
H-2	The funds can become locked in the DelegationHandler	Resolved
M-1	The fee calculation is incorrect in the _withdraw function	Resolved
M-2	Excess fees can be generated when submitting BERAs	Resolved
I-1	WBERA withdrawal cover is selected from gross balance before fee accrual	Resolved
I-2	The DelegationHandler does not account for slashing	Acknowledged
I-3	Permissionless receiveRewards can emit misleading rewards events	Resolved
I-4	Pre-threshold withdrawal short-circuit returns a redundant state flag	Resolved
I-5	Pre-threshold withdrawals ignore operator WBERA liquidity	Acknowledged
I-6	WBERA arriving after full exit is stranded	Acknowledged

4

Findings

4.1 High Risk

A total of 2 high risk findings were identified.

[H-1] SmartOperator upgrade corrupts storage layout

SEVERITY: High

IMPACT: High

STATUS: Resolved

LIKELIHOOD: High

Target

- [SmartOperator.sol](#)

Description:

SmartOperator instances are deployed behind an upgradeable beacon, so new implementations must preserve the existing storage layout. The new `_wberaBalanceAlreadyCharged` variable is inserted before the existing `protocolFeePercentage` variable instead of being appended after it.

When the beacon is upgraded, the storage slot that previously held `protocolFeePercentage` will be interpreted as `_wberaBalanceAlreadyCharged`, while `protocolFeePercentage` will read from the next slot and default to zero. Existing operators therefore lose their configured protocol fee until it is reset, and the WBERA fee tracker is initialized with an unrelated basis-point value.

This can disable protocol fee accrual across upgraded operators and can also make WBERA accounting helpers revert while the WBERA balance is below the corrupted `_wberaBalanceAlreadyCharged` value. The issue affects every already-deployed SmartOperator proxy upgraded to this implementation.

Recommendations:

Preserve the original storage layout by keeping `protocolFeePercentage` in its current slot and appending `_wberaBalanceAlreadyCharged` after it. Add storage-layout checks to the upgrade test suite so future implementation changes fail when variables are inserted, reordered, or removed.

Berachain: Resolved with [@b4945b2792 ...](#)

Zenith: Verified.

[H-2] The funds can become locked in the DelegationHandler

SEVERITY: High

IMPACT: High

STATUS: Resolved

LIKELIHOOD: Medium

Target

- [DelegationHandler.sol](#)

Description:

In StakingPool, bufferedAssets may not always be a multiple of gwei. As a result, when a deposit causes the total deposits to exceed MAX_EFFECTIVE_BALANCE, effectiveUserDeposit may also become a non-multiple of gwei.

```
function _computeDepositAmounts(uint256 userDepositAmount)
    internal
    view
    returns (
        uint256 updatedBufferedAssets,
        uint256 effectiveUserDeposit,
        uint256 refundAmount,
        uint256 rewardsToCollect,
        uint256 wberaToCollect
    )
{
    uint256 rewardsBalance = address(_stakingRewardsVault).balance;
    uint256 wberaBalance
    = ISmartOperator(_smartOperator).availableWBERABalance();
    ▷ uint256 totalBufferedAssets = userDepositAmount + rewardsBalance
    + wberaBalance + bufferedAssets;
    uint256 maxDepositAmount = MAX_EFFECTIVE_BALANCE - totalDeposits;

    // Cap total deposit at max depositable amount
    updatedBufferedAssets = maxDepositAmount < totalBufferedAssets ?
    maxDepositAmount : totalBufferedAssets;

    // Calculate how much additional deposit we can accept beyond current
    buffer
    ▷ uint256 additionalDeposit = updatedBufferedAssets - bufferedAssets;
```



```
// If we cannot accept full user deposit, calculate refund
if (additionalDeposit < userDepositAmount) {
    effectiveUserDeposit = additionalDeposit;
    refundAmount = userDepositAmount - additionalDeposit;
} else {
```

In `DelegationHandler`, `delegatedAmountAvailable` is initially maintained as a multiple of `gwei`. In `requestDelegatedFundsWithdrawal`, it also increases only by multiples of `gwei`.

```
function requestDelegatedFundsWithdrawal()
    external payable onlyRole(DEFAULT_ADMIN_ROLE) {
        uint64 delegatedAmountUsedInGWei = uint64((delegatedAmount
        - delegatedAmountAvailable) / 1 gwei);
        uint256 delegatedAmountUsed = uint256(delegatedAmountUsedInGWei
        * 1 gwei);
        _withdrawalVault.requestWithdrawal{ value: msg.value }(pubkey,
        delegatedAmountUsedInGWei, msg.value);

        delegatedFundsPendingWithdrawal = delegatedAmountUsed;
        delegatedAmountAvailable += delegatedAmountUsed;
    }
```

However, in `depositDelegatedFunds`, `delegatedAmountAvailable` can become a non-multiple of `gwei`.

```
function depositDelegatedFunds(uint256 amount)
    external whenNotPaused onlyRole(VALIDATOR_ADMIN_ROLE) {
        uint256 balanceBefore = address(this).balance;
        IStakingPool(stakingPool).submit{ value: amount }(address(this));
        uint256 depositedAmount = balanceBefore - address(this).balance;

        delegatedAmountAvailable -= depositedAmount;
    }
```

Due to this inconsistency, the difference between `delegatedAmount` and `availableFunds` in the `undelegate` function can become as large as `gwei - 1` because of rounding, rather than just 1.

```
function undelegate() external onlyRole(DEFAULT_ADMIN_ROLE) {
    // check if funds are available. Allow 1 wei of difference due to
    rounding errors.
    uint256 availableFunds = delegatedAmountAvailable
    - delegatedFundsPendingWithdrawal;
    if (delegatedAmount - availableFunds > 1) {
```

```
        revert FundsNotAvailable();  
    }  
}
```

As a result, undelegation can become impossible, causing the delegated funds to remain permanently locked.

Recommendations:

```
function undelegate() external onlyRole(DEFAULT_ADMIN_ROLE) {  
    // check if funds are available. Allow 1 wei of difference due to  
    // rounding errors.  
    uint256 availableFunds =  
        delegatedAmountAvailable - delegatedFundsPendingWithdrawal;  
    // check if funds are available. Allow 1 gwei of difference due to  
    // rounding errors.  
    uint256 availableFunds =  
        delegatedAmountAvailable / 1 gwei - delegatedFundsPendingWithdrawal /  
        1 gwei;  
    if (delegatedAmount / 1 gwei - availableFunds > 1) {  
        revert FundsNotAvailable();  
    }  
}
```

Berachain: Resolved with [@e6e6c94ac9...](#)

Zenith: Verified.

4.2 Medium Risk

A total of 2 medium risk findings were identified.

[M-1] The fee calculation is incorrect in the `_withdraw` function

SEVERITY: Medium

IMPACT: High

STATUS: Resolved

LIKELIHOOD: High

Target

- [WithdrawalVault.sol](#)

Description:

In the `_withdraw` function, the fee calculation is incorrect.

```
function _withdraw(
    bytes memory pubkey,
    uint64 assetsInGWei,
    uint256 maxFeeToPay
)
    internal
    returns (uint256 requestId)
{
    uint256 pulledFromOperator = 0;
    if (!_isFullyExited[pubkey]) {
        uint256 operatorBalance
        = ISmartOperator(coreContracts.smartOperator).availableWBERABalance();
        uint256 cap = operatorBalance >= amountInWei ? amountInWei :
        operatorBalance;
        pulledFromOperator = (cap / 1 gwei) * 1 gwei;
    }

    (uint256 shares, bool isFullExitWithdraw, bool isShortCircuit)
    = IStakingPool(coreContracts.stakingPool)
        .notifyWithdrawalRequest(msg.sender, amountInWei,
        pulledFromOperator);
```

In `notifyWithdrawalRequest`, the corresponding shares are burned from the user, which

decreases the total supply.

```
function notifyWithdrawalRequest(
    address user,
    uint256 assets,
    uint256 coveredByOperator
)
    external
    returns (uint256, bool, bool)
{
    uint256 amountOfSharesToBurn = previewWithdraw(assets);
    _burnShares(user, amountOfSharesToBurn);

    uint256 clPortion = assets - coveredByOperator;

    if (clPortion > totalDeposits || (clPortion > 0 && (totalDeposits
    - clPortion) < minEffectiveBalance())) {
        _triggerFullExit();
        _frozenTotalAssets -= assets;
        totalDeposits = 0;
        return (amountOfSharesToBurn, true, false);
    }

    totalDeposits -= clPortion;
    return (amountOfSharesToBurn, false, false);
}
```

If this withdrawal triggers a full exit, `_triggerFullExit` is called. At that point, the total assets have not changed yet, but `fullExitQueueDropBoost` is called and calculates fees using the already-reduced total supply and unchanged total assets.

```
function fullExitQueueDropBoost() external {
    ...
    _updateEarnedWBERAFees();
}
```

Additionally, if the validator is not fully exited, total assets are decreased only by the cl amount, not by the full asset amount. Later, when `pullBeraToWithdrawalVault` is called, fees are again calculated using these incorrect total asset values.

```
function _withdraw(
    bytes memory pubkey,
    uint64 assetsInGWei,
    uint256 maxFeeToPay
)
```

```

internal
returns (uint256 requestId)
{
    ...
    if (pulledFromOperator > 0 && !isShortCircuit && !isFullExitWithdraw) {
        ISmartOperator(coreContracts.smartOperator)
            .pullBeraToWithdrawalVault(pulledFromOperator);
    } else {
        pulledFromOperator = 0;
    }
}

```

As a result, the fee calculation can be inaccurate during the withdrawal flow.

Recommendations:

```

function _withdraw(
    bytes memory pubkey,
    uint64 assetsInGWei,
    uint256 maxFeeToPay
)
internal
returns (uint256 requestId)
{
    // Zero means full exit.
    if (assetsInGWei == 0) {
        revert InvalidAmount();
    }

    uint256 amountInWei = uint256(assetsInGWei) * 1 gwei;

    ICoreContractsStorage.CoreContracts memory coreContracts
    = _factory.getCoreContracts(pubkey);

    if (!IStakingPool(coreContracts.stakingPool).isFullyExited()) {
        ISmartOperator(coreContracts.smartOperator).accrueEarnedBGTFeeds();

        ISmartOperator(coreContracts.smartOperator).accrueEarnedWBERAFees();
    }

    ...
}

```

Berachain: Resolved with [@b05499a7c3...](#)

Zenith: Verified.

[M-2] Excess fees can be generated when submitting BERAs

SEVERITY: Medium

IMPACT: High

STATUS: Resolved

LIKELIHOOD: High

Target

- [StakingPool.sol](#)

Description:

In the `_submit` function, shares corresponding to the deposit amount are minted first, and rewards are collected afterward through the `_collectRewards` function.

```
function _submit(address receiver)
    internal whenNotPaused returns (uint256 shares) {
    if (userDepositAmount > 0) {
        shares = previewDeposit(userDepositAmount);
        _mintShares(receiver, shares);
    }

    _collectRewards(rewardsToCollect, wberaToCollect);

    bufferedAssets = updatedBufferedAssets;
```

The issue is that the total supply increases due to the newly minted shares, while the total assets are not yet updated when the fee is calculated.

```
function mintFeeShares(uint256 amount) external {
    _validateSender(_smartOperator);

    uint256 shares = previewDeposit(amount);
    _mintShares(_defaultSharesRecipient, shares);
}
```

As a result, the share price becomes lower than the correct value, causing more fee shares to be minted than intended. This leads to dilution and causes depositors to incur losses due to the excess fee minting.

Recommendations:

```
function _submit(address receiver)
    internal whenNotPaused returns (uint256 shares) {

    if (userDepositAmount > 0) {
        shares = previewDeposit(userDepositAmount);
        _mintShares(receiver, shares);
    }

    _collectRewards(rewardsToCollect, wberaToCollect);

    if (shares > 0) {
        _mintShares(receiver, shares);
    }
}
```

Berachain: Resolved with [@b0cf65a19f ...](#)

Zenith: Verified.

4.3 Informational

A total of 6 informational findings were identified.

[I-1] WBERA withdrawal cover is selected from gross balance before fee accrual

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [WithdrawalVault.sol](#)

Description:

`WithdrawalVault._withdraw()` computes the operator-covered withdrawal amount from `SmartOperator.availableWBERABalance()`, which returns the full WBERA balance held by the operator. That value is passed to `StakingPool.notifyWithdrawalRequest()` as `coveredByOperator`, causing the pool to decrement `totalDeposits` only by assets - `coveredByOperator`.

However, `StakingPool._getTotalAssets()` accounts for operator WBERA through `SmartOperator.rebaseableWberaAmount()`, which excludes pending protocol fees from the WBERA balance. This creates an ordering mismatch between the liquidity used to cover withdrawals and the asset amount used to price the shares being burned. The withdrawal cover is selected from gross physical WBERA before fees are crystallized, while share pricing uses fee-net WBERA.

The current flow does not show a direct withdrawal shortfall because `SmartOperator.pullBeraToWithdrawalVault()` accrues WBERA fees before unwrapping and transferring funds. The low-severity impact is on accountability and fee-share timing: when a withdrawal triggers this accrual, fee shares are minted after `notifyWithdrawalRequest()` has already burned the withdrawing user's shares. This can under-mint protocol fee shares compared with accruing the pending WBERA fee before the withdrawal burn, leaving the difference with remaining stBERA holders.

Recommendations:

Explicitly accrue WBERA fees before reading `availableWBERABalance()` and before `notifyWithdrawalRequest()` burns shares. After fees are materialized, the withdrawal vault can continue using `availableWBERABalance()` as the operator cover while keeping fee-share accounting aligned with the withdrawal pricing path.

Berachain: Resolved with [@b05499a7c3 ...](#)

Zenith: Verified.

[I-2] The DelegationHandler does not account for slashing

SEVERITY: Informational

IMPACT: Informational

STATUS: Acknowledged

LIKELIHOOD: Low

Target

- [DelegationHandler.sol](#)

Description:

The total deposits can decrease due to slashing, which lowers the share price.

```
function updateTotalDeposits(uint256 newTotalDeposits)
    external whenNotPaused {
    if (newTotalDeposits == 0) {
        _triggerFullExit();
        IWithdrawalVault(_withdrawalVault).notifyFullExitFromCL(_pubkey);
    } else {
        totalDeposits = newTotalDeposits;
    }
}
```

However, the DelegationHandler does not account for this scenario. It always attempts to withdraw the originally deposited amount.

```
function requestDelegatedFundsWithdrawal()
    external payable onlyRole(DEFAULT_ADMIN_ROLE) {
    uint256 balanceBefore = address(this).balance - msg.value;

    uint64 delegatedAmountUsedInGWei = uint64((delegatedAmount
    - delegatedAmountAvailable) / 1 gwei);
    uint256 delegatedAmountUsed = uint256(delegatedAmountUsedInGWei
    * 1 gwei);
    _withdrawalVault.requestWithdrawal{ value: msg.value }(pubkey,
    delegatedAmountUsedInGWei, msg.value);
}
```

If the total assets have decreased because of slashing, the withdrawal can revert, making undelegation impossible. Additionally, when the share price decreases, this behavior can also consume protocol fees unexpectedly.

Recommendations:

Track the minted shares instead. Alternatively, in the `requestDelegatedFundsWithdrawal` function, calculate the maximum withdrawable amount and use the minimum value between that amount and the original deposited amount.

Berachain: Acknowledged. The slashing mechanic is not yet implemented on Berachain consensus and is not currently on the roadmap.

[I-3] Permissionless receiveRewards can emit misleading rewards events

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [StakingPool.sol](#)

Description:

`StakingPool.receiveRewards()` is now callable by any address and emits `StakingRewardsReceived(msg.value)` without checking that the sender is the `StakingRewardsVault` or the `SmartOperator`. The transferred native BERA is not added to `bufferedAssets`, and `StakingPool._getTotalAssets()` does not include the staking pool's raw native balance, so arbitrary calls do not directly inflate share value or affect normal on-chain deposit and withdrawal accounting.

The main risk is off-chain ambiguity. Any caller can send native BERA directly to `receiveRewards()` and produce a `StakingRewardsReceived` event that looks identical to a legitimate reward collection event from `StakingRewardsVault.withdrawRewards()` or `SmartOperator.withdrawRewards()`. If monitoring, analytics, accounting, or rewards attribution systems rely on this event without validating the transaction sender and surrounding state changes, they can overstate protocol rewards or misclassify arbitrary donations as earned staking rewards.

The donated native BERA remains untracked surplus in the staking pool until a full exit forwards the pool's raw native balance to the withdrawal vault. This does not create a direct user claim through share accounting, but it can further confuse reconciliation between emitted rewards events, accounting state, and actual contract balances.

Recommendations:

Restrict `receiveRewards()` to the expected reward sources, currently the staking rewards vault and smart operator. If permissionless donations are intended, emit a separate donation event or include the sender in the event and document that `StakingRewardsReceived` is not sufficient proof of protocol-earned rewards.

Berachain: Resolved with [@f003dcf82c ...](#)

Zenith: Verified.

[I-4] Pre-threshold withdrawal short-circuit returns a redundant state flag

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [StakingPool.sol](#)

Description:

`StakingPool.notifyWithdrawalRequest()` returns `(amountOfSharesToBurn, isFullyExited, true)` from the pre-threshold short-circuit branch. That branch is only reachable after the function has already handled and returned from the `isFullyExited` case, and it is inside the `!activeThresholdReached` path, so `isFullyExited` must be false at that point.

Returning the storage variable is currently equivalent to returning `false`, so there is no direct withdrawal accounting impact. However, the return value controls how `WithdrawalVault._withdraw()` decides whether to skip consensus-layer withdrawal logic and refund the execution-layer fee, so leaving it tied to a mutable state read makes the control-flow contract less explicit than necessary.

Recommendations:

Return `false` directly from the pre-threshold short-circuit branch. This documents the invariant in the code and avoids making the withdrawal vault depend on a state value that cannot validly be true in that path.

Berachain: Resolved with [@5774ad72f0...](#)

Zenith: Verified.

[I-5] Pre-threshold withdrawals ignore operator WBERA liquidity

SEVERITY: Informational

IMPACT: Informational

STATUS: Acknowledged

LIKELIHOOD: Low

Target

- [StakingPool.sol](#)
- [WithdrawalVault.sol](#)

Description:

`WithdrawalVault._withdraw()` pre-computes operator WBERA cover and passes it to `StakingPool.notifyWithdrawalRequest()` as `coveredByOperator`. This cover is used in the post-threshold withdrawal path to reduce the CL-bound portion of the withdrawal, but the pre-threshold branch ignores it entirely.

Before `activeThresholdReached`, `StakingPool.notifyWithdrawalRequest()` only short-circuits if `bufferedAssets ≥ assets`. If the native buffer is smaller than the requested withdrawal but the operator has enough WBERA to cover the shortfall, the request still reverts with `WithdrawalNotAllowed()`. This is inconsistent with the rest of the WBERA refactor because operator WBERA is included in `totalAssets()` through `rebaseableWberaAmount()` and is treated as withdrawal cover after the activation threshold.

This does not directly create a solvency shortfall, but it can block withdrawals whose share value is backed by WBERA already counted in pool accounting. It also leaves the `WithdrawalVault` and `StakingPool` handshake ambiguous: the vault computes a WBERA cover hint for all non-full-exited validators, while the pool silently discards that hint in the pre-threshold state.

Recommendations:

Define whether operator WBERA should be usable as pre-threshold withdrawal liquidity. If it should, update the pre-threshold branch to consider `bufferedAssets + coveredByOperator` and ensure the vault still pulls the WBERA when the pool returns a short-circuit result. If pre-threshold withdrawals must only use native buffer, avoid computing or passing WBERA cover in that state and document the restriction explicitly.

Berachain: Acknowledged.

Pre-threshold WBERA inflows are not expected by design. Before `activeThresholdReached`, the validator is not yet on the consensus layer and has no protocol-driven path to accrue WBERA. WBERA is a standard ERC20, so a third party can transfer it directly to the operator, but we treat that as an unsolicited donation rather than a supported funding mode. The contracts still handle it correctly: such WBERA counts toward `totalAssets` via `rebaseableWberaAmount` (raising share value for all holders), and any user can migrate it into `bufferedAssets` permissionlessly by calling `StakingPool.processRewards()`, after which `requestWithdrawal` succeeds through the existing pre-threshold short-circuit. Operator-WBERA cover is therefore intentionally not exposed as a withdrawal source in the pre-threshold branch — that branch is scoped to the native buffer to keep early-lifecycle accounting simple. The `coveredByOperator` hint computed by the vault is meaningful only post-activation, by design. No funds are stranded and no privileged action is required.

[I-6] WBERA arriving after full exit is stranded

SEVERITY: Informational

IMPACT: Informational

STATUS: Acknowledged

LIKELIHOOD: Low

Target

- [SmartOperator.sol](#)

Description:

`StakingPool._triggerFullExit()` collects the operator's current WBERA balance once, unwraps it through `SmartOperator.withdrawRewards()`, and forwards the resulting native BERA to the `WithdrawalVault`. After the pool is marked fully exited, `StakingPool.totalAssets()` returns the frozen asset value and normal reward-processing paths are no longer available.

If WBERA arrives at the `SmartOperator` after that full-exit transition, the current post-exit recovery path does not collect it. `WithdrawalVault._finalizeWithdrawalRequest()` only attempts `SmartOperator.fullExitRedeemBGT()` for fully exited validators, and `fullExitRedeemBGT()` redeems BGT but does not unwrap WBERA. The normal `WithdrawalVault._withdraw()` WBERA cover path is also skipped once the vault marks the pubkey as fully exited.

This can leave late-arriving WBERA stranded in the operator. It is also not recoverable through `moveERC20ToIncentiveCollector()` because that function intentionally skips WBERA as a pool underlying asset.

Recommendations:

Add a full-exit WBERA drain path that can be called by the withdrawal vault, either by extending `fullExitRedeemBGT()` to also unwrap and send WBERA or by adding a dedicated `fullExitRedeemWBERA()` function. Ensure the post-exit path updates WBERA fee tracking consistently before unwrapping and define whether late WBERA should increase withdrawal-vault liquidity or be otherwise accounted separately.

Berachain: Acknowledged.

Late-arriving WBERA after a full exit is not expected to be material by design. Once `_triggerFullExit()` runs, the validator is on its way out of the active set and has no sustained path to accrue WBERA: `processRewards`, `accrueEarnedWBERAFees`, `withdrawRewards`, and the vault's `pullBeraToWithdrawalVault` cover branch are all gated

by the `pause / whenNotFullyExited / _isFullyExited[pubkey]` checks. The only protocol-driven inflow that can still occur is BeraChef rewards in the window between `fullExitQueueDropBoost` and actual departure from the active set, where the validator only proposes probabilistically (one slot per epoch among the full active set) and the window is bounded by the CL exit queue. Each produced block pays roughly 0.4 WBERA to the operator, so the expected residual is on the order of a few payouts. The other sources — sub-gwei rounding dust from the `gwei-floor` in `_withdraw()` and accidental external transfers — are treated as unsolicited and out of scope.

BGT is handled differently, and `fullExitRedeemBGT` is intentionally scoped to it: the BGT held at the operator at exit time is already accounted for in pool NAV (fees have been charged on it via `_updateEarnedBGTfees` along the way), but it cannot be redeemed in a single step because the drop-boost is queued and must clear before `BGT.dropBoost(...)` settles and the balance becomes redeemable. The two-step flow — `fullExitQueueDropBoost` at exit, then `fullExitRedeemBGT` once the queue has elapsed, invoked lazily on the first `_finalizeWithdrawalRequest` — exists to bridge that queue delay. WBERA has no equivalent queueing constraint, so the same two-step recovery hook isn't needed for it.

Post-exit accounting for WBERA is intentionally frozen: `_frozenTotalAssets` is captured at the transition, so any late WBERA recovered to the pool would not mint fee shares (accrual is gated) and would not raise per-share NAV (frozen). Exposing a recovery path would therefore require either unfreezing NAV or adding a separate surplus-distribution mechanism on the vault — both materially larger changes than the residual amount justifies. Operationally, pending rewards are expected to be processed before triggering the full-exit transition, which closes most of the window. No funds critical to shareholder accounting are stranded and no privileged action is required to preserve solvency.